



UNSW
SYDNEY

COMP3153/9153

Algorithmic Verification

Lecture 5: Ordered Binary Decision Diagrams

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$                                 /* Atomic Proposition */
2   | return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$                              /* Negation */
4   | return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$                              /* Exists Next */
6   | return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$                              /* Always Next */
8   | return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$            /* Exists Until */
10  | return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$            /* Always Until */
12  | return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$                              /* Exists Globally */
14  | return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$                              /* Exists Eventually */
16  | return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\varphi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\varphi) \cup \delta^{-1}(X)$ 
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\varphi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\varphi) \cup \delta^{-1}(X)$ 
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 
```

Procedure FixPoint(f, ι)

```
1  $P := \iota$ ;
2  $N := f(P)$ ;
3 while  $P \neq N$  do
4    $P := N$ ;
5    $N := f(N)$ ;
6 return  $N$ ;
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```
1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 
```

Procedure FixPoint(f, ι)

```
1  $P := \iota$ ;
2  $N := f(P)$ ;
3 while  $P \neq N$  do
4    $P := N$ ;
5    $N := f(N)$ ;
6 return  $N$ ;
```

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```

1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E } \psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\psi_2) \cup (\text{SMC}(\psi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A } \psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\psi_2) \cup (\text{SMC}(\psi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 

```

Procedure FixPoint(f, ι)

```

1  $P := \iota$ ;
2  $N := f(P)$ ;
3 while  $P \neq N$  do
4    $P := N$ ;
5    $N := f(N)$ ;
6 return  $N$ ;

```

$$A \models \varphi \iff q_0 \models \varphi$$

$$\iff q_0 \in \text{SMC}(\varphi)$$

Symbolic CTL Model Checking Algorithm

$A = (Q, q_0, \delta, L)$ fixed

Procedure SMC(φ)

```

1 case  $\varphi \in \mathcal{P}$  /* Atomic Proposition */
2   return  $L^{-1}(\varphi)$ ;
3 case  $\varphi = \neg\psi$  /* Negation */
4   return  $Q \setminus \text{SMC}(\psi)$ ;
5 case  $\varphi = \text{EX}\psi$  /* Exists Next */
6   return  $\delta^{-1}(\text{SMC}(\psi))$ ;
7 case  $\varphi = \text{AX}\psi$  /* Always Next */
8   return  $Q \setminus \delta^{-1}(Q \setminus \text{SMC}(\psi))$ ;
9 case  $\varphi = \text{E}\psi_1 \text{ UNTIL } \psi_2$  /* Exists Until */
10  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap \delta^{-1}(X))$ 
11 case  $\varphi = \text{A}\psi_1 \text{ UNTIL } \psi_2$  /* Always Until */
12  return  $\mu X. \text{SMC}(\varphi_2) \cup (\text{SMC}(\varphi_1) \cap (Q \setminus \delta^{-1}(Q \setminus X)))$ 
13 case  $\varphi = \text{EG}\varphi$  /* Exists Globally */
14  return  $\nu X. \text{SMC}(\psi) \cap \delta^{-1}(X)$ 
15 case  $\varphi = \text{EF}\varphi$  /* Exists Eventually */
16  return  $\mu X. \text{SMC}(\psi) \cup \delta^{-1}(X)$ 

```

Procedure FixPoint(f, ι)

```

1  $P := \iota$ ;
2  $N := f(P)$ ;
3 while  $P \neq N$  do
4    $P := N$ ;
5    $N := f(N)$ ;
6 return  $N$ ;

```

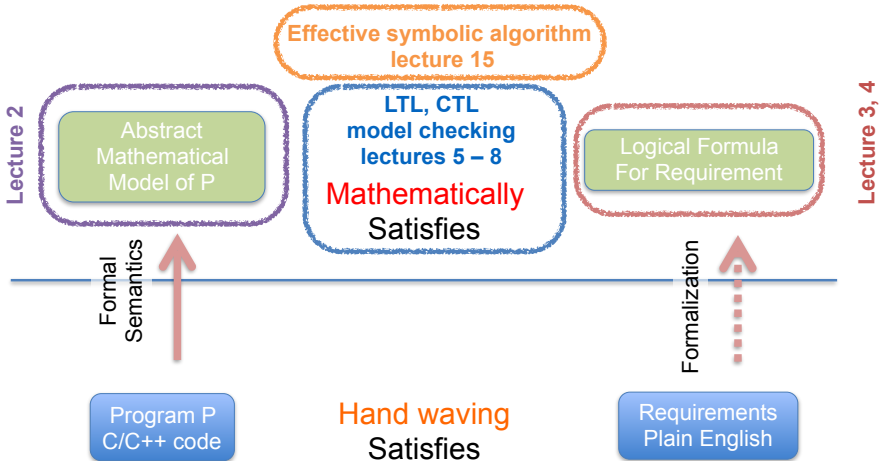
$$A \models \varphi \iff q_0 \models \varphi$$

$$\iff q_0 \in \text{SMC}(\varphi)$$

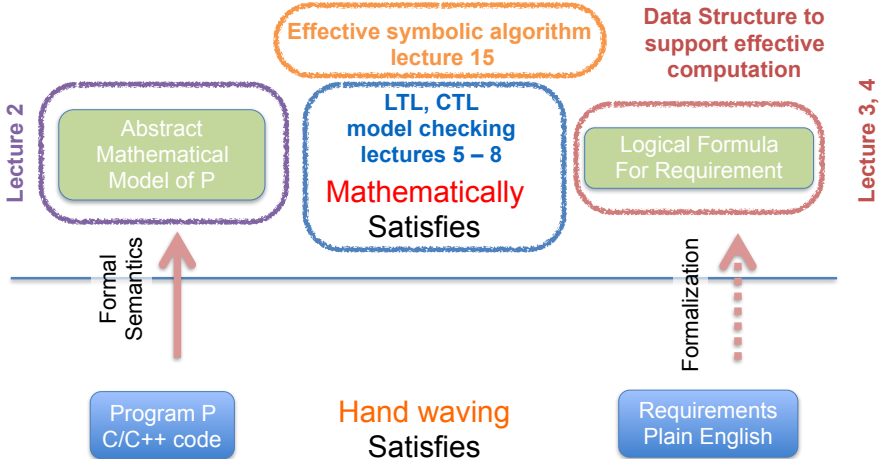
Set operations:

- $\cap, \cup, \setminus$
- Equality test
- δ, δ^{-1}

The Big Picture



The Big Picture



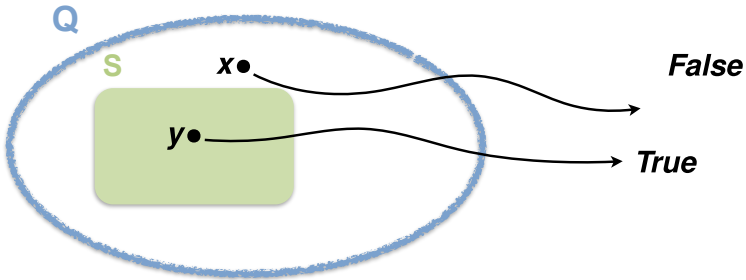
Content for this lecture

1. Sets as Boolean functions
2. Binary Decision Diagrams (BDDs)
3. Operations on BDDs
4. Properties and transition relation
5. Model Checking with BDDs

Sets as Boolean Functions

Characteristic function of a set

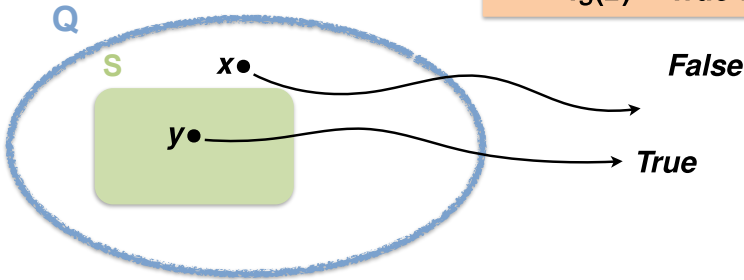
Characteristic Functions



Characteristic Functions

Characteristic function f_S of S

- $f_S(z) = \text{False}$ for $z \notin S$
- $f_S(z) = \text{True}$ for $z \in S$



Boolean Encoding

Finite set E

Let $n = \log_2 |E|$

Boolean Encoding

Finite set E

Given $S \subseteq E$:

Let $n = \log_2 |E|$

Boolean Encoding

Finite set E

Given $S \subseteq E$:

Let $n = \log_2 |E|$

$$f_S : \mathbb{B}^n \longrightarrow \mathbb{B}$$

Boolean Encoding

Finite set E

Given $S \subseteq E$:

Let $n = \log_2 |E|$

$$f_S : \mathbb{B}^n \longrightarrow \mathbb{B}$$

$E = \{0, 1, 2, 3, 4, 5, 6, 7\}$

3 bits: b_3, b_2, b_1

Boolean Encoding

Finite set E

Given $S \subseteq E$:

Let $n = \log_2 |E|$

$$f_S : \mathbb{B}^n \longrightarrow \mathbb{B}$$

$E = \{0, 1, 2, 3, 4, 5, 6, 7\}$

3 bits: b_3, b_2, b_1

	b_3	b_2	b_1
6	1	1	0
3	0	1	1

Boolean Encoding

Finite set E

Given $S \subseteq E$:

Let $n = \log_2 |E|$

$$f_S : \mathbb{B}^n \longrightarrow \mathbb{B}$$

$E = \{0, 1, 2, 3, 4, 5, 6, 7\}$

3 bits: b_3, b_2, b_1

	b_3	b_2	b_1
6	1	1	0
3	0	1	1

$S = \{k \mid k \bmod 2 = 0\}$

$$f_S : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

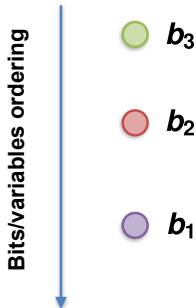
$$f_S(b_1, b_2, b_3) = (b_1 = 0)$$

Ordered Binary Decision Trees

$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_s : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

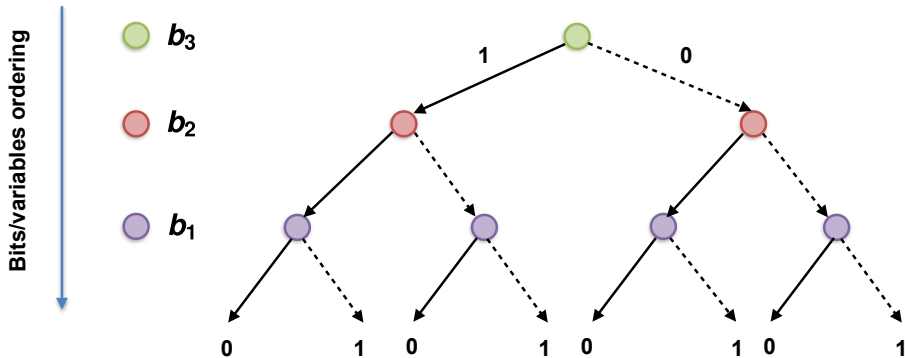
Ordered Binary Decision Trees



$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_s : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

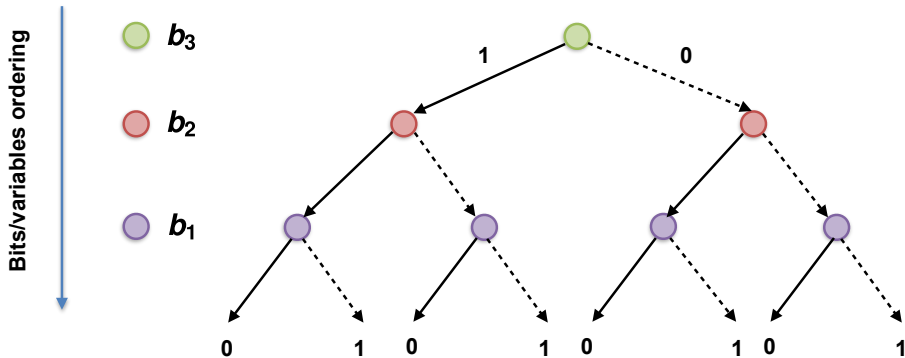
Ordered Binary Decision Trees



$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_s : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

Ordered Binary Decision Trees

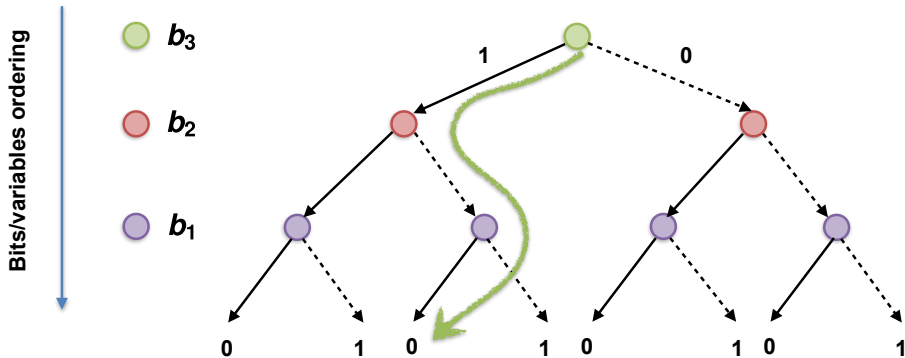


$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_s : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

$$f_s(b_1, b_2, b_3) = (b_1 = 0)$$

Ordered Binary Decision Trees

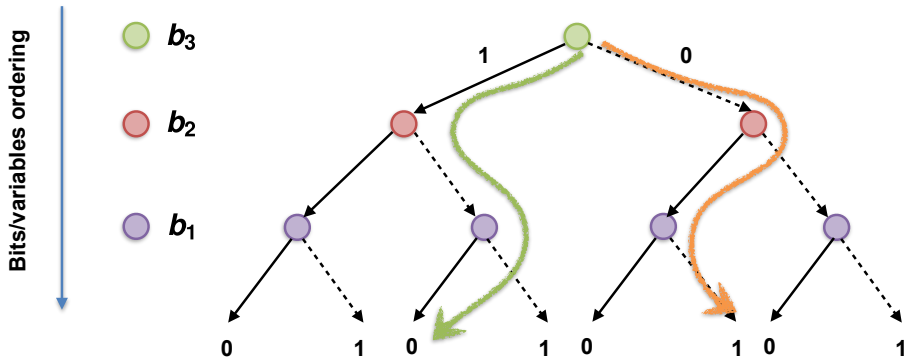


$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_S : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

$$f_S(b_1, b_2, b_3) = (b_1 = 0)$$

Ordered Binary Decision Trees

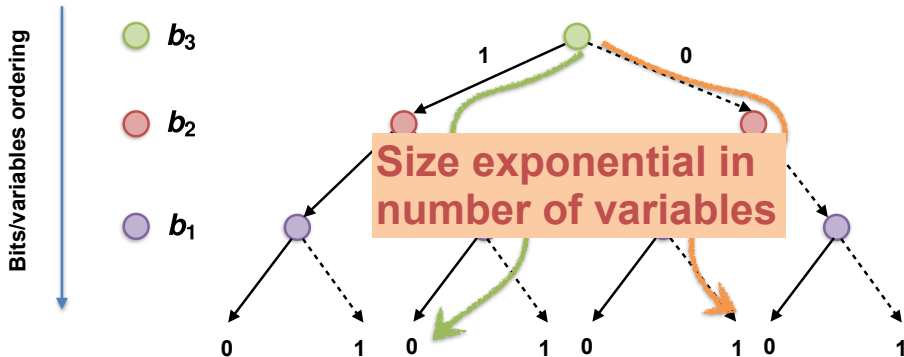


$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_S : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

$$f_S(b_1, b_2, b_3) = (b_1 = 0)$$

Ordered Binary Decision Trees



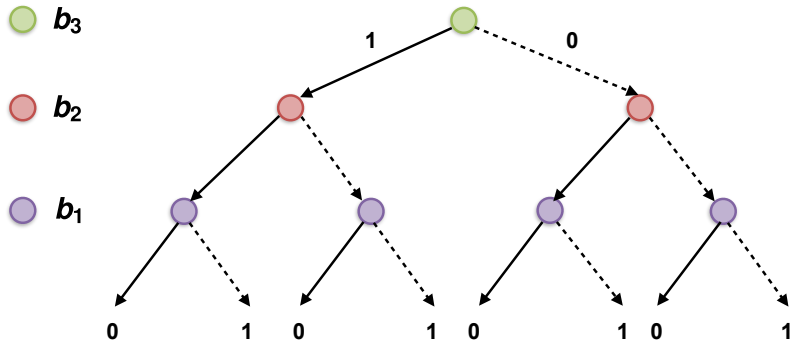
$$S = \{k \mid k \bmod 2 = 0\}$$

$$f_S : \mathbb{B}^3 \longrightarrow \mathbb{B}$$

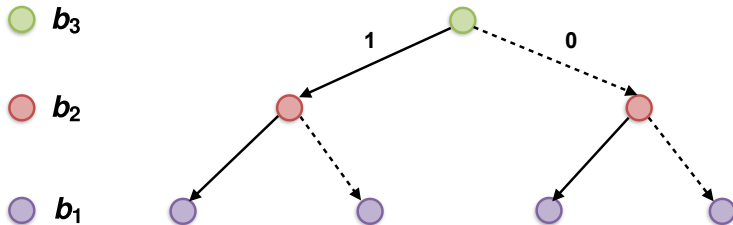
$$f_S(b_1, b_2, b_3) = (b_1 = 0)$$

Binary Decision Diagrams

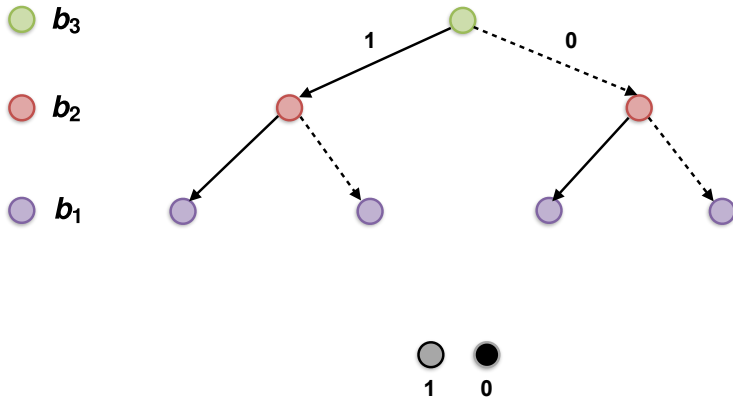
Ordered (Reduced) Binary Decision Diagram



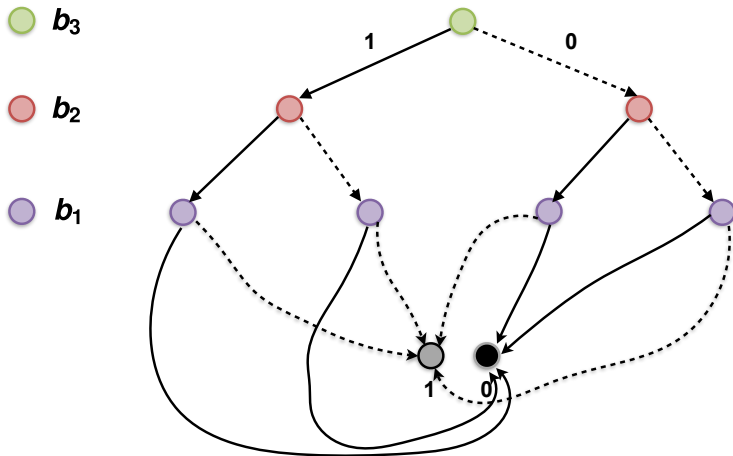
Ordered (Reduced) Binary Decision Diagram



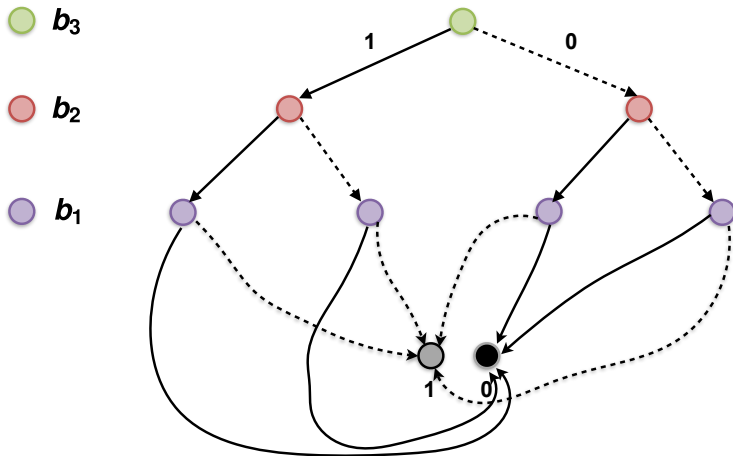
Ordered (Reduced) Binary Decision Diagram



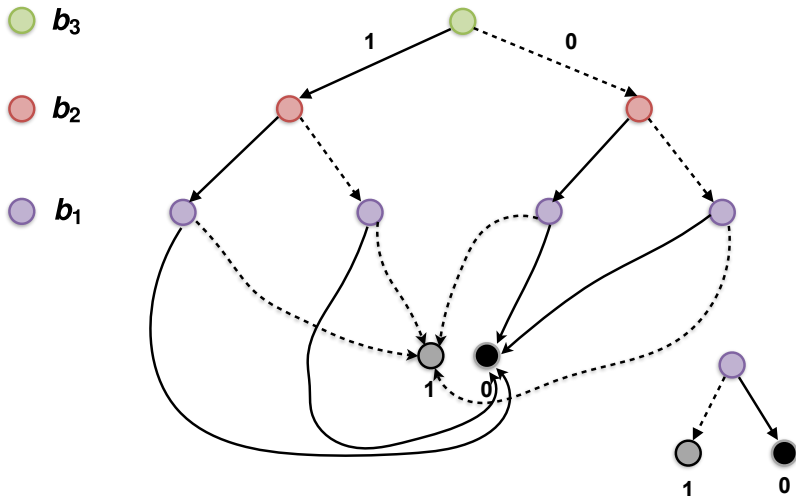
Ordered (Reduced) Binary Decision Diagram



Ordered (Reduced) Binary Decision Diagram



Ordered (Reduced) Binary Decision Diagram

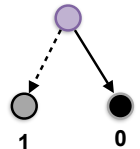
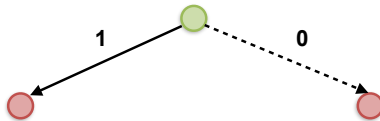


Ordered (Reduced) Binary Decision Diagram

 b_3

 b_2

 b_1

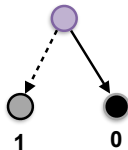
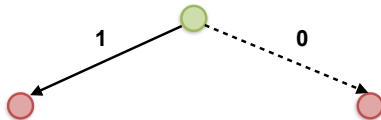


Ordered (Reduced) Binary Decision Diagram

 b_3

 b_2

 b_1

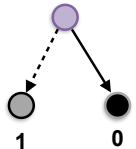
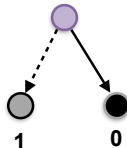
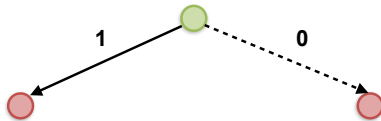


Ordered (Reduced) Binary Decision Diagram

 b_3

 b_2

 b_1

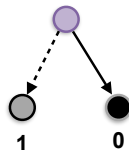
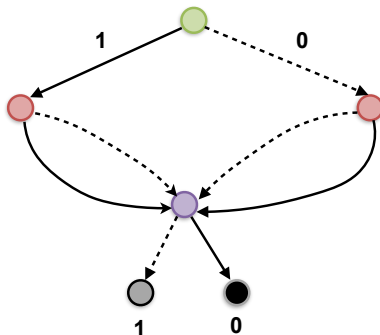


Ordered (Reduced) Binary Decision Diagram

 b_3

 b_2

 b_1

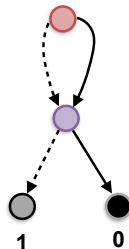
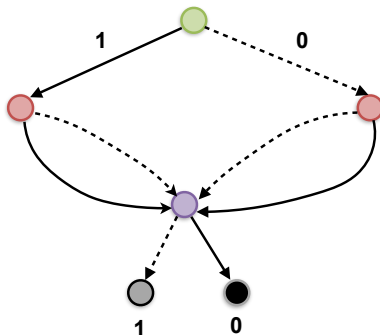


Ordered (Reduced) Binary Decision Diagram

● b_3

● b_2

● b_1

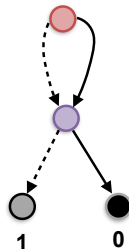
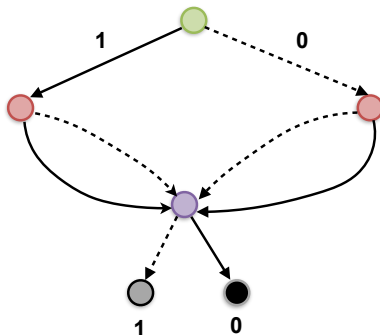


Ordered (Reduced) Binary Decision Diagram

● b_3

● b_2

● b_1



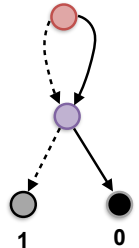
Ordered (Reduced) Binary Decision Diagram

 b_3



 b_2

 b_1

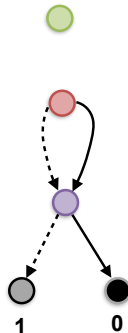


Ordered (Reduced) Binary Decision Diagram

● b_3

● b_2

● b_1

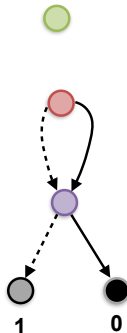


Ordered (Reduced) Binary Decision Diagram

 b_3

 b_2

 b_1

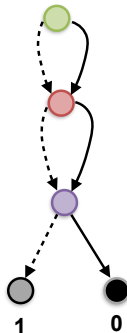


Ordered (Reduced) Binary Decision Diagram

● b_3

● b_2

● b_1

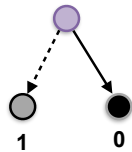
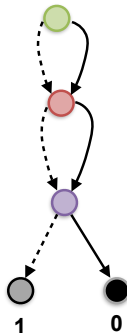


Ordered (Reduced) Binary Decision Diagram

● b_3

● b_2

● b_1



O(R)BDD

Variables Order (1)

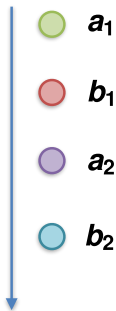
Example: two-bit comparator [Clarke et al.]

$$a_1 = b_1 \wedge a_2 = b_2$$

Variables Order (1)

Example: two-bit comparator [Clarke et al.]

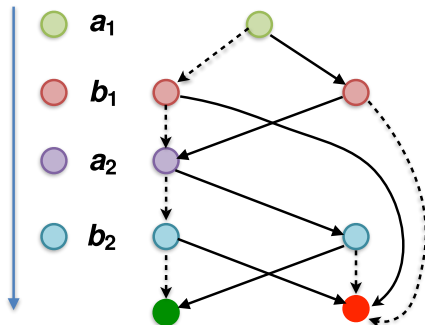
$$a_1 = b_1 \wedge a_2 = b_2$$



Variables Order (1)

Example: two-bit comparator [Clarke et al.]

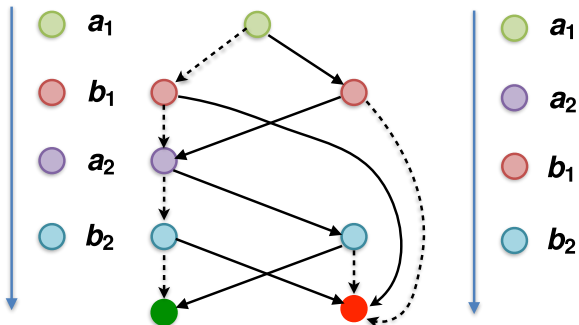
$$a_1 = b_1 \wedge a_2 = b_2$$



Variables Order (1)

Example: two-bit comparator [Clarke et al.]

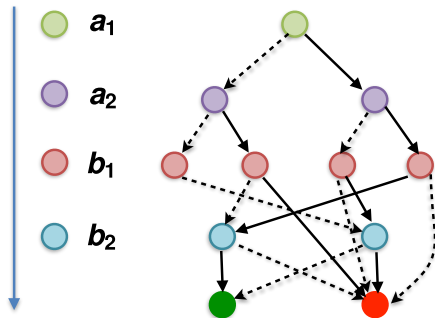
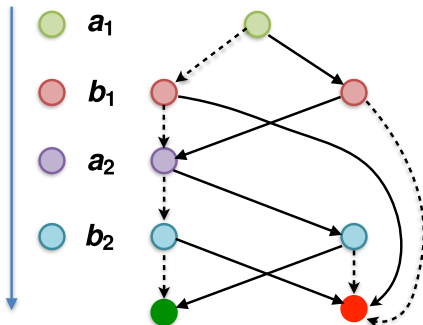
$$a_1 = b_1 \wedge a_2 = b_2$$



Variables Order (1)

Example: two-bit comparator [Clarke et al.]

$$a_1 = b_1 \wedge a_2 = b_2$$



Variables Order (2)

1. Finding an **optimal** order is **NP-complete**
2. There are **boolean functions** that have **exponential size BDDs** regardless of order

Solutions:

- **smart order** (related variables close in BDD)
- **dynamic re-ordering**

Properties of ORBDDs

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$

Properties of ORBDDs

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$

- fixed order for the variables
- maximally reduced

Properties of ORBDDs

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$

- fixed order for the variables
- maximally reduced

$BDD(f(\bar{b}))$ is uniquely defined

Properties of ORBDDs

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$

- fixed order for the variables
- maximally reduced

$BDD(f(\bar{b}))$ is uniquely defined

or equivalently: there is a canonical form for **$BDD(f(\bar{b}))$**

Properties of ORBDDs

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

- fixed order for the variables
- maximally reduced

$BDD(f(\bar{b}))$ is uniquely defined

or equivalently: there is a canonical form for **$BDD(f(\bar{b}))$**

Operations on BDDs

Encoding Boolean Functions

Complement

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Complement

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\neg f(\bar{b})$

Complement

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\neg f(\bar{b})$

Example: two-bit comparator [Clarke et al.] $a_1 = b_1 \wedge a_2 = b_2$

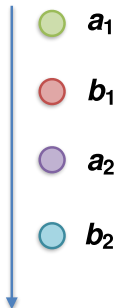
Complement

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\neg f(\bar{b})$

Example: two-bit comparator [Clarke et al.] $a_1 = b_1 \wedge a_2 = b_2$



Complement

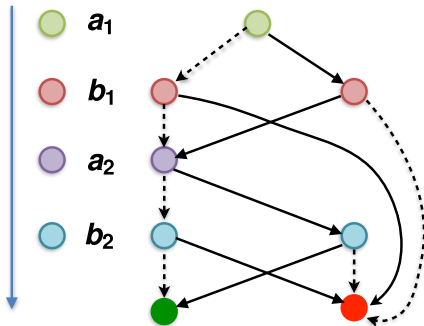
Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\neg f(\bar{b})$

Example: two-bit comparator [Clarke et al.]

$$a_1 = b_1 \wedge a_2 = b_2$$



Complement

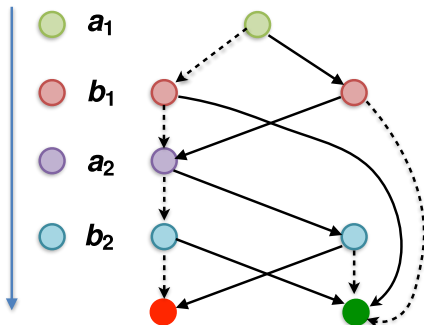
Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\neg f(\bar{b})$

Example: two-bit comparator [Clarke et al.]

$$a_1 = b_1 \wedge a_2 = b_2$$



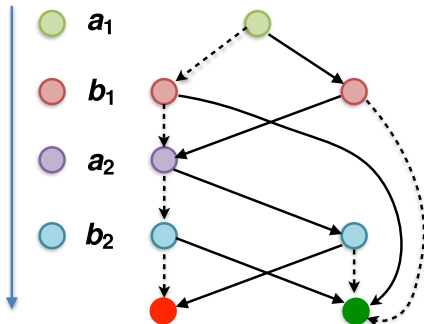
Complement

Fix an ordering $b_1 < b_2 < \dots < b_k$

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\neg f(\bar{b})$

Example: two-bit comparator [Clarke et al.] $a_1 = b_1 \wedge a_2 = b_2$



$$BDD(\neg f(\bar{b})) = \overline{BDD(f(\bar{b}))}$$

Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Restrict

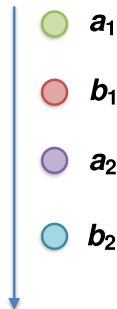
Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

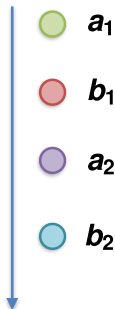
Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$



Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

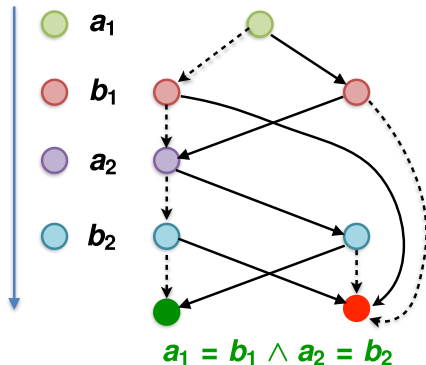


$$a_1 = b_1 \wedge a_2 = b_2$$

Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

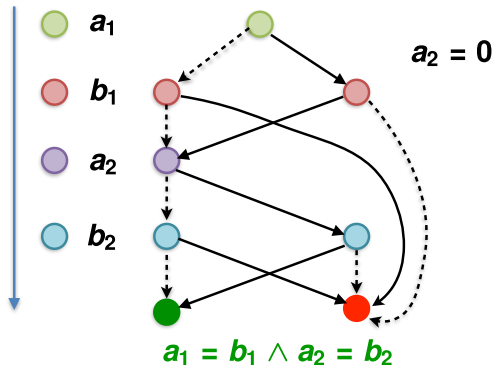
Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$



Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

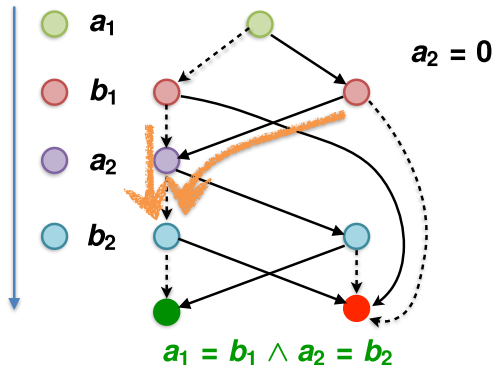
Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$



Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

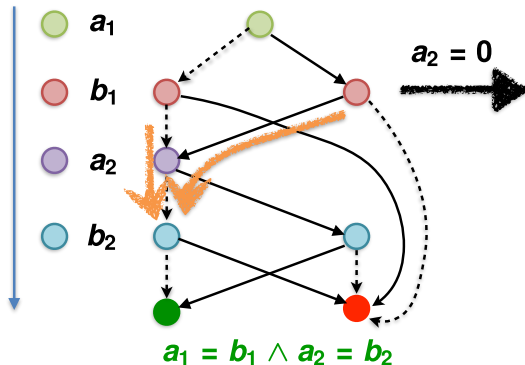


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

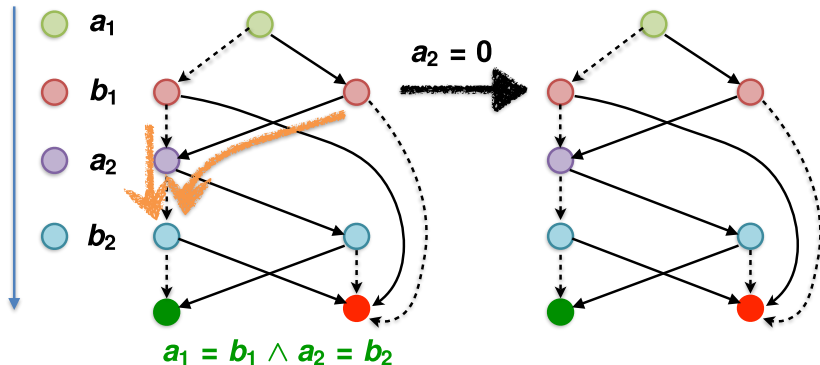
Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$



Restrict

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

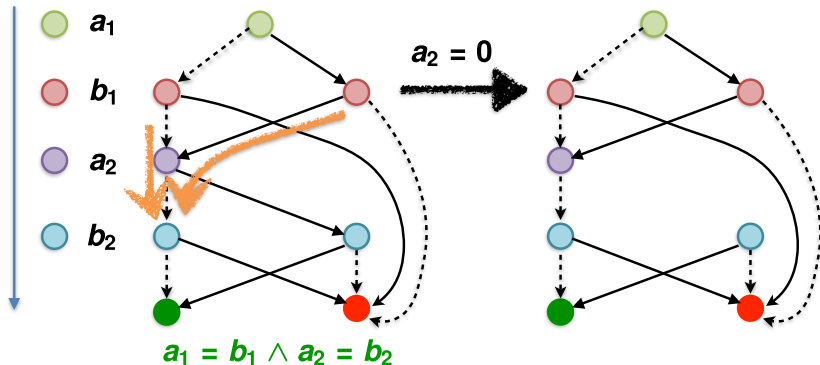


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

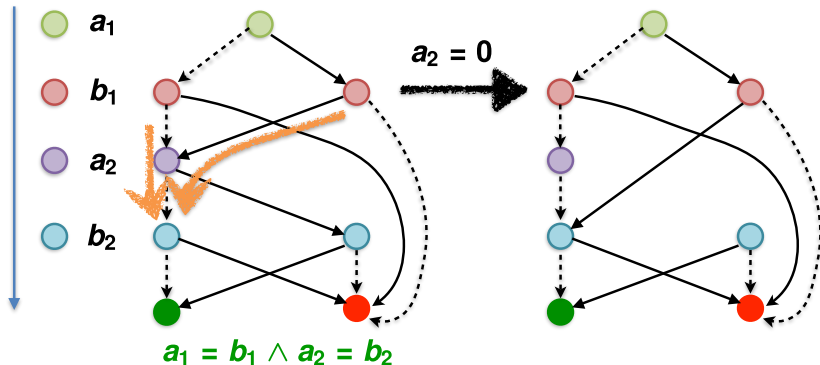


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

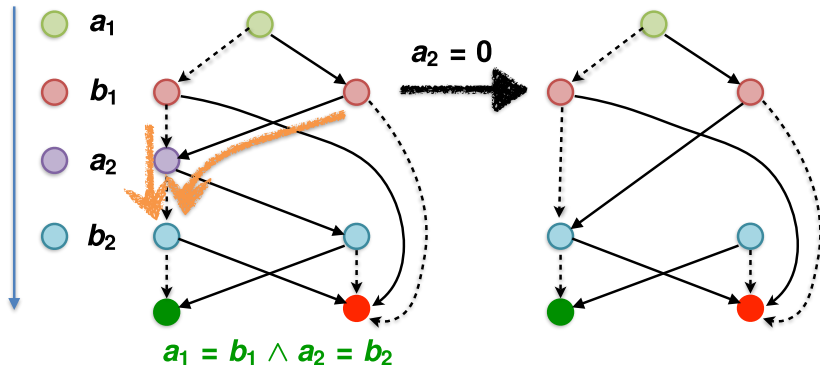


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

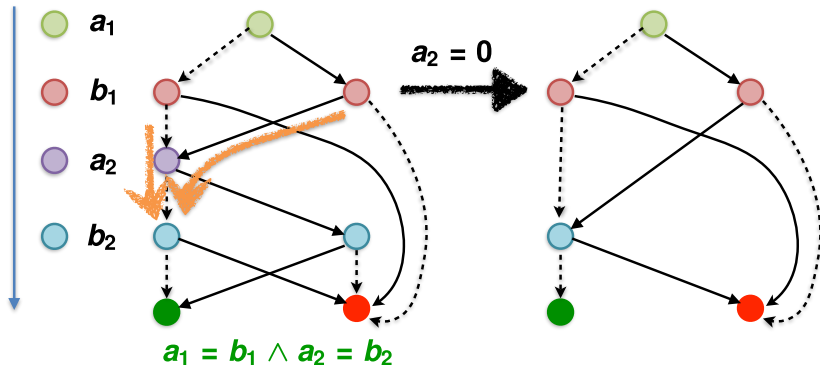


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

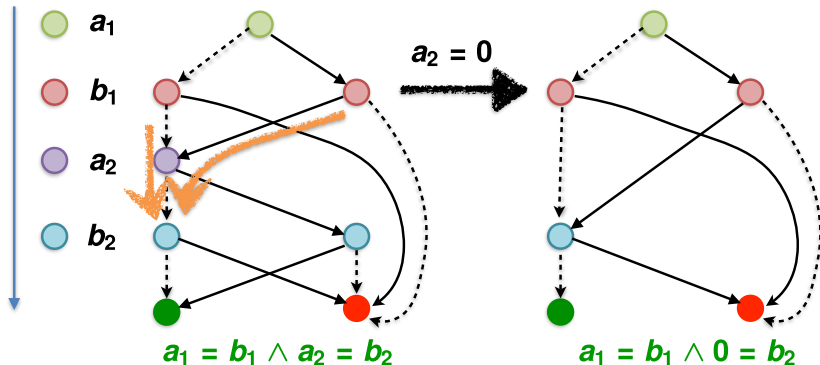


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

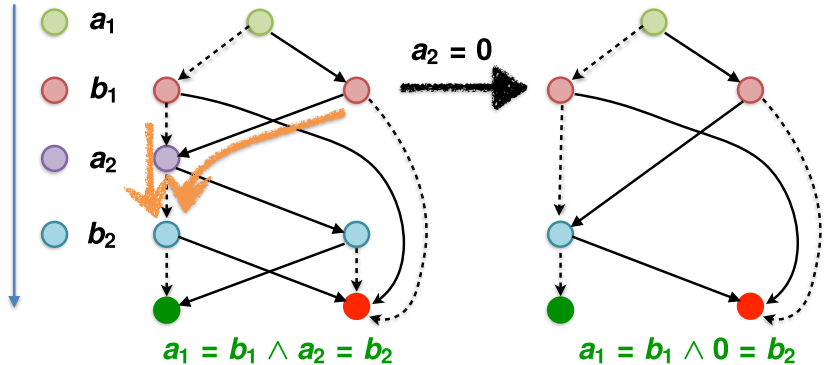


Restrict

Boolean function $f(\bar{b})$

$BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: Build BDD for $f(\bar{b})[b_j/0]$ or $f(\bar{b})[b_j/1]$

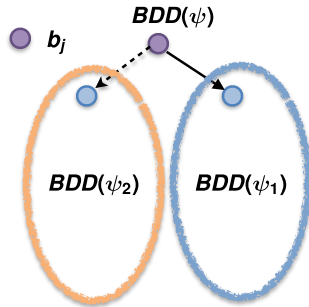
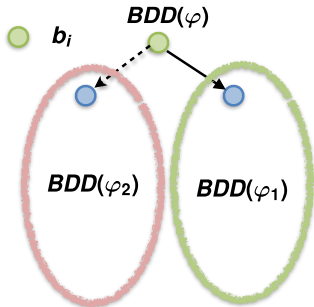


$$BDD(f(\bar{b})[b_j/v]) = \text{Restrict}(BDD(f(\bar{b})), b_j, v)$$

And – Intersection

Fix an ordering $b_1 < b_2 < \dots < b_k$

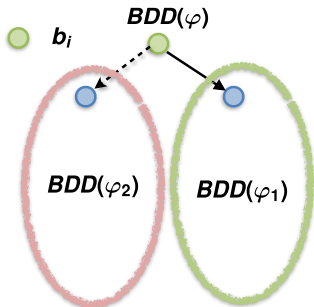
Goal: build BDD for $\varphi \wedge \psi$



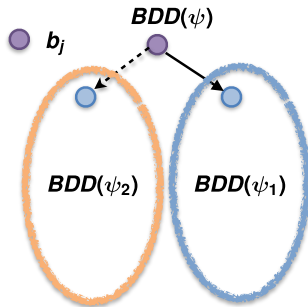
And – Intersection

Fix an ordering $b_1 < b_2 < \dots < b_k$

Goal: build BDD for $\varphi \wedge \psi$



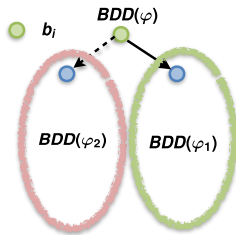
$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



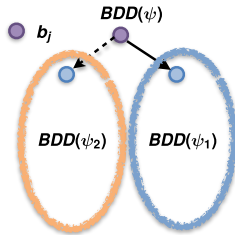
$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

And – Intersection

Fix an ordering $b_1 < b_2 < \dots < b_k$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$

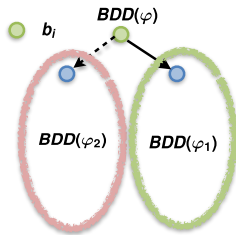


$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

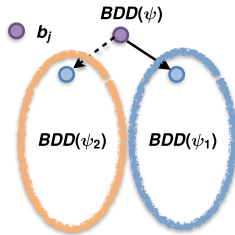
And – Intersection

Fix an ordering $b_1 < b_2 < \dots < b_k$

$i > j$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



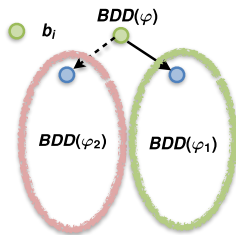
$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

And – Intersection

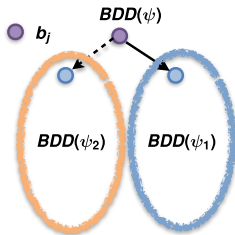
Fix an ordering $b_1 < b_2 < \dots < b_k$

$i > j$

$\varphi \wedge \psi = \text{if } b_j \text{ then } \varphi_1 \wedge \psi \text{ else } \varphi_2 \wedge \psi$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



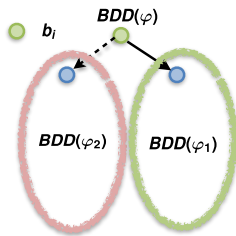
$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

And – Intersection

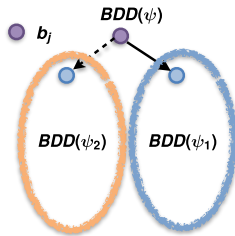
Fix an ordering $b_1 < b_2 < \dots < b_k$

$i > j$

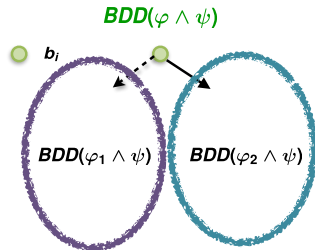
$\varphi \wedge \psi = \text{if } b_j \text{ then } \varphi_1 \wedge \psi \text{ else } \varphi_2 \wedge \psi$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$

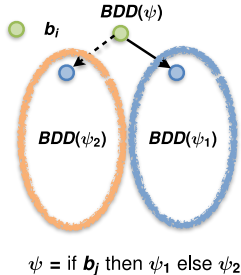
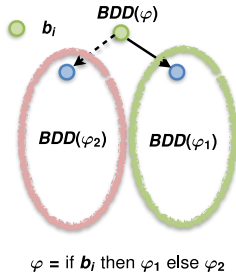


$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$



And – Intersection

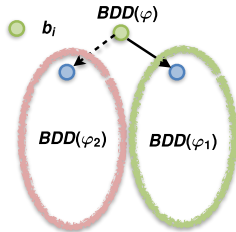
Fix an ordering $b_1 < b_2 < \dots < b_k$



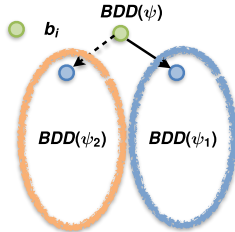
And – Intersection

Fix an ordering $b_1 < b_2 < \dots < b_k$

$i = j$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



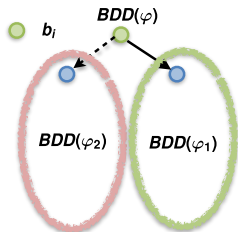
$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

And – Intersection

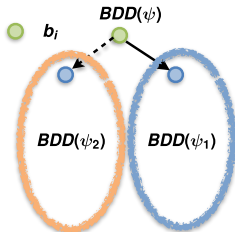
Fix an ordering $b_1 < b_2 < \dots < b_k$

$i = j$

$\varphi \wedge \psi = \text{if } b_i \text{ then } \varphi_1 \wedge \psi_1 \text{ else } \varphi_2 \wedge \psi_2$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



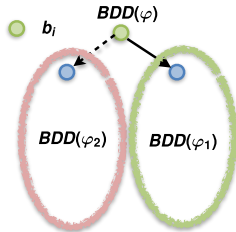
$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

And – Intersection

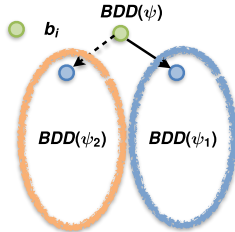
Fix an ordering $b_1 < b_2 < \dots < b_k$

$i = j$

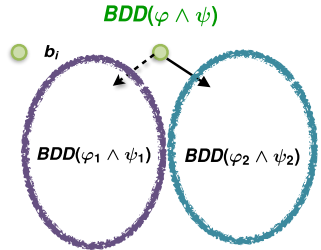
$\varphi \wedge \psi = \text{if } b_j \text{ then } \varphi_1 \wedge \psi_1 \text{ else } \varphi_2 \wedge \psi_2$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$

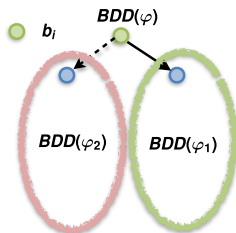


And – Intersection

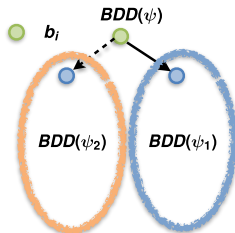
Fix an ordering $b_1 < b_2 < \dots < b_k$

$i = j$

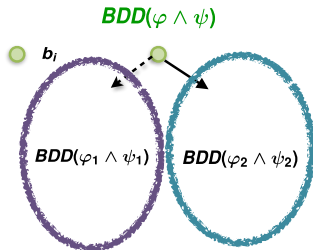
$\varphi \wedge \psi = \text{if } b_j \text{ then } \varphi_1 \wedge \psi_1 \text{ else } \varphi_2 \wedge \psi_2$



$\varphi = \text{if } b_i \text{ then } \varphi_1 \text{ else } \varphi_2$



$\psi = \text{if } b_j \text{ then } \psi_1 \text{ else } \psi_2$



$$BDD(f_1(\bar{b}) \wedge f_2(\bar{b})) = BDD(f_1(\bar{b})) \otimes BDD(f_2(\bar{b}))$$

OR – Union

Boolean functions $f_1(\bar{\mathbf{b}})$ and $f_2(\bar{\mathbf{b}})$ $\mathbf{BDD}(f_i(\bar{\mathbf{b}}))$ the ORBDD for $f_i(\bar{\mathbf{b}})$

Goal: build BDD for $f_1(\bar{\mathbf{b}}) \vee f_2(\bar{\mathbf{b}})$

OR – Union

Boolean functions $f_1(\bar{b})$ and $f_2(\bar{b})$ **BDD**($f_i(\bar{b})$) the ORBDD for $f_i(\bar{b})$

Goal: build BDD for $f_1(\bar{b}) \vee f_2(\bar{b})$

$$f_1(\bar{b}) \vee f_2(\bar{b}) = \neg(\neg f_1(\bar{b}) \wedge \neg f_2(\bar{b}))$$

OR – Union

Boolean functions $f_1(\bar{b})$ and $f_2(\bar{b})$ **BDD($f_i(\bar{b})$)** the ORBDD for $f_i(\bar{b})$

Goal: build BDD for $f_1(\bar{b}) \vee f_2(\bar{b})$

$$f_1(\bar{b}) \vee f_2(\bar{b}) = \neg(\neg f_1(\bar{b}) \wedge \neg f_2(\bar{b}))$$

$$\mathbf{BDD}(f_1(\bar{b}) \vee f_2(\bar{b})) = \mathbf{BDD}(\neg(\neg f_1(\bar{b}) \wedge \neg f_2(\bar{b})))$$

OR – Union

Boolean functions $f_1(\bar{b})$ and $f_2(\bar{b})$ **$BDD(f_i(\bar{b}))$** the ORBDD for $f_i(\bar{b})$

Goal: build BDD for $f_1(\bar{b}) \vee f_2(\bar{b})$

$$f_1(\bar{b}) \vee f_2(\bar{b}) = \neg(\neg f_1(\bar{b}) \wedge \neg f_2(\bar{b}))$$

$$BDD(f_1(\bar{b}) \vee f_2(\bar{b})) = BDD(\neg(\neg f_1(\bar{b}) \wedge \neg f_2(\bar{b})))$$

$$BDD(f_1(\bar{b})) \oplus BDD(f_2(\bar{b})) = \overline{(BDD(f_1(\bar{b})) \otimes BDD(f_2(\bar{b})))}$$

Existential Quantification

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\exists b_j. f(\bar{b})$

Existential Quantification

Boolean function $f(\bar{b})$ $BDD(f(\bar{b}))$ the ORBDD for $f(\bar{b})$

Goal: build BDD for $\exists b_j. f(\bar{b})$

$$\exists b_j. f(\bar{b}) = f(\bar{b})[b_j/0] \vee f(\bar{b})[b_j/1]$$

Existential Quantification

Boolean function $f(\bar{b})$ **BDD**($f(\bar{b})$) the ORBDD for $f(\bar{b})$

Goal: build BDD for $\exists b_j.f(\bar{b})$

$$\exists b_j.f(\bar{b}) = f(\bar{b})[b_j/0] \vee f(\bar{b})[b_j/1]$$

$$BDD(\exists b_j.f(\bar{b})) = BDD(f(\bar{b})[b_j/0]) \oplus BDD(f(\bar{b})[b_j/1])$$

Existential Quantification

Boolean function $f(\bar{b})$ **$BDD(f(\bar{b}))$** the ORBDD for $f(\bar{b})$

Goal: build BDD for $\exists b_j. f(\bar{b})$

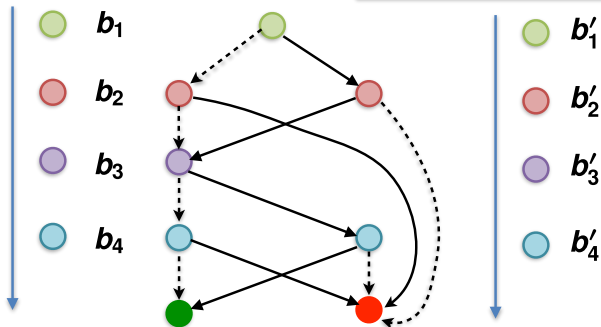
$$\exists b_j. f(\bar{b}) = f(\bar{b})[b_j/0] \vee f(\bar{b})[b_j/1]$$

$$BDD(\exists b_j. f(\bar{b})) = BDD(f(\bar{b})[b_j/0]) \oplus BDD(f(\bar{b})[b_j/1])$$

$$\exists b_j. BDD(f(\bar{b})) = BDD(f(\bar{b})[b_j/0]) \oplus BDD(f(\bar{b})[b_j/1])$$

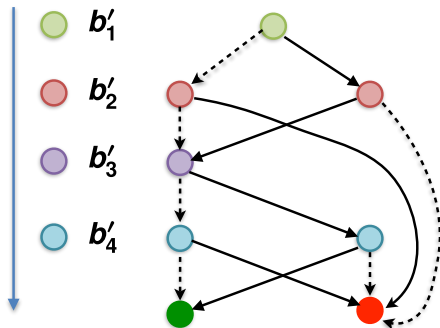
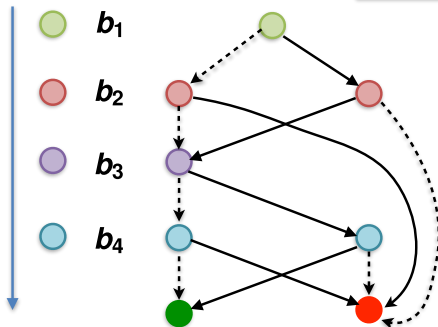
Renaming

Rename $\bar{b}$: $\bar{b}'$



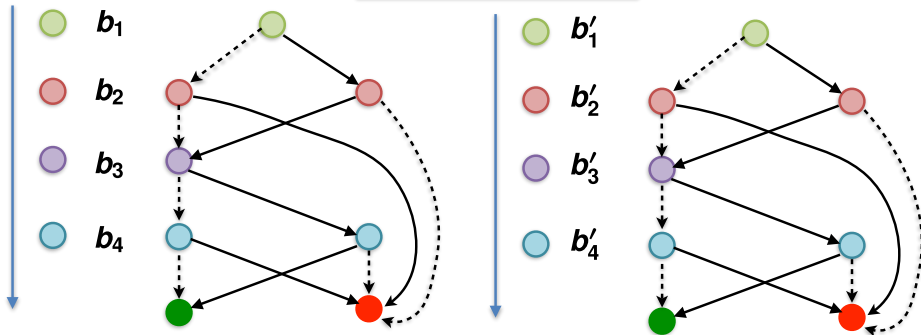
Renaming

Rename $\bar{b}$: $\bar{b}'$



Renaming

Rename $\bar{b} : \bar{b}'$



$$\text{Rename}[\bar{b} / \bar{b}'](BDD(f(\bar{b}))) = BDD(f[\bar{b} / \bar{b}'])$$

Operations on BDDs

$\overline{BDD(bdd_1)}$

swap leaves **0** and **1**

Restrict(bdd_1, b_j, v)

Instantiate b_j with v

$bdd_1 \otimes bdd_2$

Conjunction

$bdd_1 \oplus bdd_2$

Disjunction

$\exists b_j. bdd_1$

Existential quantification on b_j

Rename[b/b'](bdd_1)

Variables renaming

Properties & Transition Relation

Encoding with BDDs

Atomic Properties as Boolean Formulas

$$\mathbf{A} = (\mathbf{Q}, q_0, \delta, L)$$

$$p \in \mathcal{P}, \llbracket p \rrbracket = L^{-1}(p)$$

Atomic Properties as Boolean Formulas

$$A = (Q, q_0, \delta, L)$$

$$p \in \mathcal{P}, \llbracket p \rrbracket = L^{-1}(p)$$

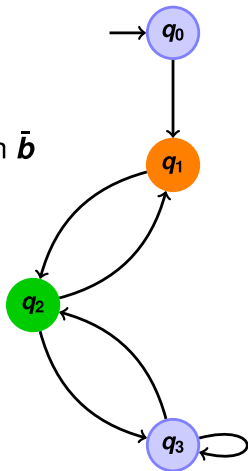
Let $\text{BinEnc}(v, \bar{b})$ = binary encoding of v on $\bar{b}$

Atomic Properties as Boolean Formulas

$$A = (Q, q_0, \delta, L)$$

$$p \in \mathcal{P}, \llbracket p \rrbracket = L^{-1}(p)$$

Let $\text{BinEnc}(v, \bar{b})$ = binary encoding of v on $\bar{b}$



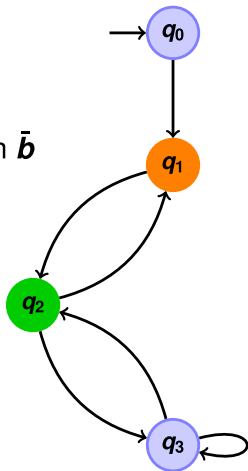
Atomic Properties as Boolean Formulas

$$A = (Q, q_0, \delta, L)$$

$$p \in \mathcal{P}, \llbracket p \rrbracket = L^{-1}(p)$$

Let $\text{BinEnc}(v, \bar{b}) = \text{binary encoding of } v \text{ on } \bar{b}$

$$f_{\llbracket \bullet \rrbracket} \equiv b_2 = 0 \wedge b_1 = 1$$



Atomic Properties as Boolean Formulas

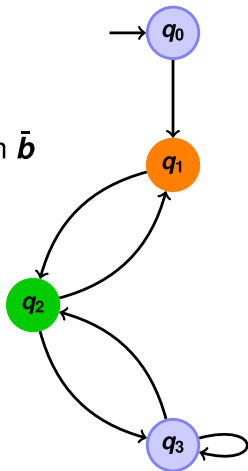
$$A = (Q, q_0, \delta, L)$$

$$p \in \mathcal{P}, \llbracket p \rrbracket = L^{-1}(p)$$

Let $\text{BinEnc}(v, \bar{b}) = \text{binary encoding of } v \text{ on } \bar{b}$

$$f_{\llbracket \bullet \rrbracket} \equiv b_2 = 0 \wedge b_1 = 1$$

$$f_{\llbracket \bullet \rrbracket} \equiv b_2 = 1 \wedge b_1 = 0$$



Atomic Properties as Boolean Formulas

$$A = (Q, q_0, \delta, L)$$

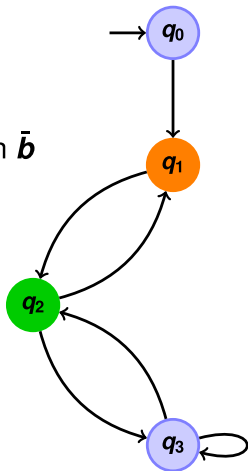
$$p \in \mathcal{P}, \llbracket p \rrbracket = L^{-1}(p)$$

Let $\text{BinEnc}(v, \bar{b}) = \text{binary encoding of } v \text{ on } \bar{b}$

$$f_{\llbracket \bullet \rrbracket} \equiv b_2 = 0 \wedge b_1 = 1$$

$$f_{\llbracket \bullet \rrbracket} \equiv b_2 = 1 \wedge b_1 = 0$$

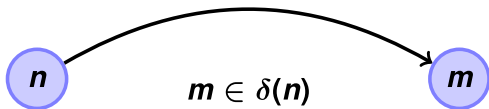
$$f_{\llbracket \bullet \rrbracket} \equiv (b_2 = b_1 = 0) \vee (b_2 = b_1 = 1)$$



One Transition as Boolean Formula

$$A = (Q, q_0, \delta, L)$$

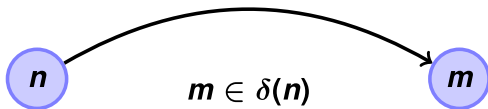
$$\delta \subseteq Q \times Q$$



One Transition as Boolean Formula

$$A = (Q, q_0, \delta, L)$$

$$\delta \subseteq Q \times Q$$

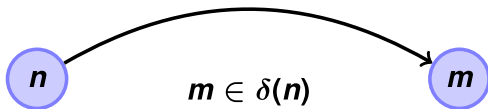


$$f_\delta(n, m) = \text{BinEnc}(n, \bar{b}) \wedge \text{BinEnc}(m, \bar{b}')$$

One Transition as Boolean Formula

$$A = (Q, q_0, \delta, L)$$

$$\delta \subseteq Q \times Q$$



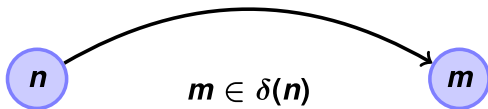
$$f_\delta(n, m) = \text{BinEnc}(n, \bar{b}) \wedge \text{BinEnc}(m, \bar{b}')$$

	b_3	b_2	b_1		b'_3	b'_2	b'_1
6	1	1	0	3	0	1	1

One Transition as Boolean Formula

$$A = (Q, q_0, \delta, L)$$

$$\delta \subseteq Q \times Q$$



$$f_\delta(n, m) = \text{BinEnc}(n, \bar{b}) \wedge \text{BinEnc}(m, \bar{b}')$$

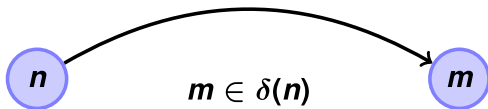
	b_3	b_2	b_1		b'_3	b'_2	b'_1
6	1	1	0	3	0	1	1

$$n = 6, m = 3$$

One Transition as Boolean Formula

$$A = (Q, q_0, \delta, L)$$

$$\delta \subseteq Q \times Q$$



$$f_\delta(n, m) = \text{BinEnc}(n, \bar{b}) \wedge \text{BinEnc}(m, \bar{b}')$$

	b_3	b_2	b_1		b'_3	b'_2	b'_1
6	1	1	0	3	0	1	1

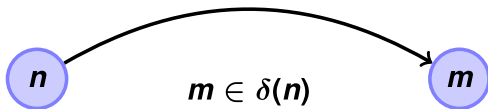
$$n = 6, m = 3$$

$$b_3 \wedge b_2 \wedge \neg b_1 \wedge \neg b'_3 \wedge b'_2 \wedge b'_1$$

One Transition as Boolean Formula

$$A = (Q, q_0, \delta, L)$$

$$\delta \subseteq Q \times Q$$



$$f_\delta(n, m) = \text{BinEnc}(n, \bar{b}) \wedge \text{BinEnc}(m, \bar{b}')$$

	b_3	b_2	b_1
6	1	1	0

	b'_3	b'_2	b'_1
3	0	1	1

$$n = 6, m = 3$$

$$k = \log_2 |Q|$$

$$b_3 \wedge b_2 \wedge \neg b_1 \wedge \neg b'_3 \wedge b'_2 \wedge b'_1$$

$$2 \cdot k \text{ variables } \bar{b}, \bar{b}'$$

Transition Relation as Boolean Formula

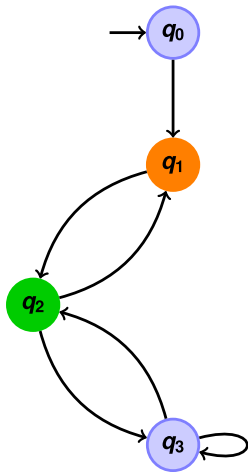
$$\mathbf{f}_\delta = \bigvee_{n \in Q, m \in \delta(n)} \mathbf{f}_\delta(n, m)$$

Transition Relation as Boolean Formula

$$f_{\delta} = \bigvee_{n \in Q, m \in \delta(n)} f_{\delta}(n, m)$$

$$f_{\delta}(\bar{b}, \bar{b}') = \textit{True} \iff \bar{b}' \in \delta(\bar{b})$$

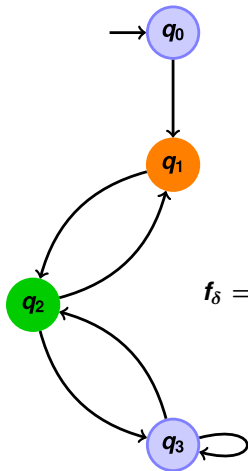
Transition Relation as Boolean Formula



$$f_{\delta} = \bigvee_{n \in Q, m \in \delta(n)} f_{\delta}(n, m)$$

$$f_{\delta}(\bar{b}, \bar{b}') = \text{True} \iff \bar{b}' \in \delta(\bar{b})$$

Transition Relation as Boolean Formula



$$f_{\delta} = \bigvee_{n \in Q, m \in \delta(n)} f_{\delta}(n, m)$$

$$f_{\delta}(\bar{b}, \bar{b}') = \text{True} \iff \bar{b}' \in \delta(\bar{b})$$

$$f_{\delta} = \begin{array}{ll} (b_2 = b_1 = 0 \wedge b'_2 = 0 \wedge b'_1 = 1) & (q_0, q_1) \\ \vee (b_2 = 0 \wedge b_1 = 1 \wedge b'_2 = 1 \wedge b'_1 = 0) & (q_1, q_2) \\ \vee (b_2 = 1 \wedge b_1 = 0 \wedge b'_2 = 0 \wedge b'_1 = 1) & (q_2, q_1) \\ \vee (b_2 = 1 \wedge b_1 = 0 \wedge b'_2 = b'_1 = 1) & (q_2, q_3) \\ \vee (b_2 = b_1 = 1 \wedge b'_2 = 1 \wedge b'_1 = 0) & (q_3, q_2) \\ \vee (b_2 = b_1 = 1 \wedge b'_2 = b'_1 = 1) & (q_3, q_3) \end{array}$$

Encoding Image

$$\mathbf{A} = (\mathbf{Q}, q_0, \delta, L)$$

Goal: compute BDD for $\delta(X)$

Encoding Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

Encoding Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

$$\delta \quad f_\delta(\bar{b}, \bar{b}') \quad bdd_\delta$$

Encoding Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

$$\delta \quad f_\delta(\bar{b}, \bar{b}') \quad bdd_\delta$$

$$f_{\delta(X)} = (\exists \bar{b}. (f_X(\bar{b}) \wedge f_\delta))[\bar{b}' / \bar{b}]$$

Encoding Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

$$\delta \quad f_\delta(\bar{b}, \bar{b}') \quad bdd_\delta$$

$$f_{\delta(X)} = (\exists \bar{b}. (f_X(\bar{b}) \wedge f_\delta))[\bar{b}' / \bar{b}]$$

$$bdd_{\delta(X)} = \text{Rename}[\bar{b}' / \bar{b}](\exists \bar{b}. (bdd_X \otimes bdd_\delta))$$

Encoding Pre-Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta^{-1}(X)$

Encoding Pre-Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta^{-1}(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

Encoding Pre-Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta^{-1}(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

$$\delta \quad f_\delta(\bar{b}, \bar{b}') \quad bdd_\delta$$

Encoding Pre-Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta^{-1}(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

$$\delta \quad f_\delta(\bar{b}, \bar{b}') \quad bdd_\delta$$

$$f_{\delta^{-1}(X)} = \exists \bar{b}'. (f_X[\bar{b}/\bar{b}'] \wedge f_\delta)$$

Encoding Pre-Image

$$A = (Q, q_0, \delta, L)$$

Goal: compute BDD for $\delta^{-1}(X)$

$$X \quad f_X(\bar{b}) \quad bdd_X$$

$$\delta \quad f_\delta(\bar{b}, \bar{b}') \quad bdd_\delta$$

$$f_{\delta^{-1}(X)} = \exists \bar{b}'. (f_X[\bar{b}/\bar{b}'] \wedge f_\delta)$$

$$bdd_{\delta^{-1}(X)} = \exists \bar{b}'. (\text{Rename}[\bar{b}/\bar{b}'](bdd_X) \otimes bdd_\delta)$$

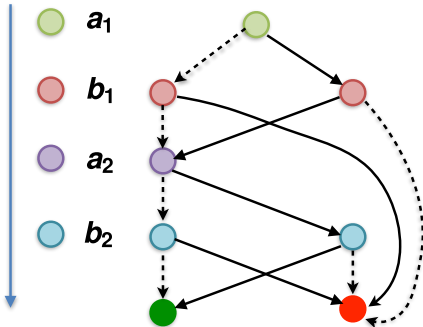
Model Checking with BDDs

Checking Equality

BDD(False) = ●

BDD(True) = ●

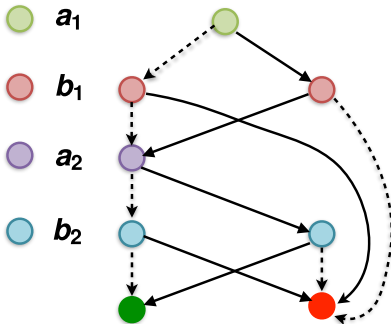
$$a_1 = b_1 \wedge a_2 = b_2$$



Checking Equality

BDD(False) = ●

BDD(True) = ●



$$a_1 = b_1 \wedge a_2 = b_2$$

Checking Equality

$BDD(False) =$ ●

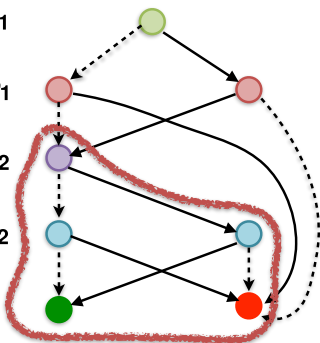
$BDD(True) =$ ●

● a_1

● b_1

● a_2

● b_2

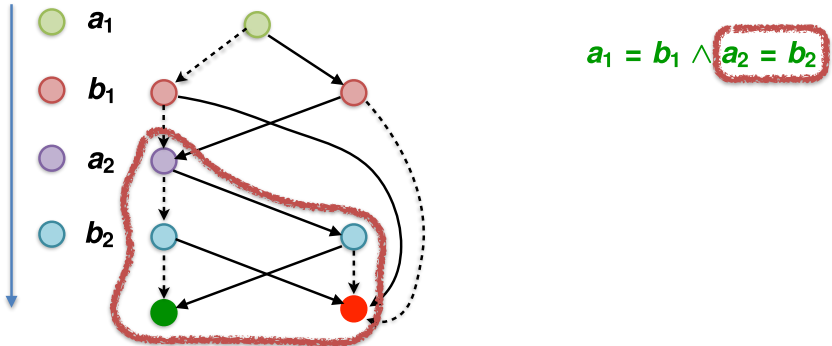


$a_1 = b_1 \wedge a_2 = b_2$

Checking Equality

$BDD(False) =$ ●

$BDD(True) =$ ●

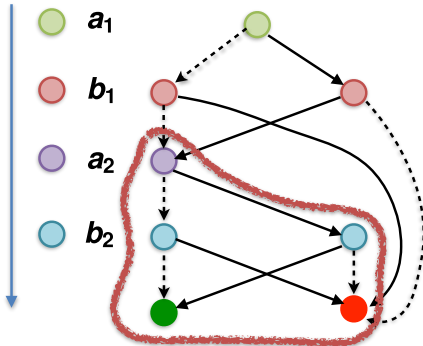


Each BDD **uniquely** represented in memory

Checking Equality

$BDD(False) =$ ●

$BDD(True) =$ ●



$$a_1 = b_1 \wedge a_2 = b_2$$

Consequences:

- Checking **equality**: **constant time**
- **Hash table** to store previously computed BDDs

Each BDD **uniquely** represented in memory

BDD-based Model Checking Algorithm

Given: $A = (Q, q_0, \delta, L)$ and $\varphi \in CTL$

BDD-based Model Checking Algorithm

Given: $\mathbf{A} = (\mathbf{Q}, q_0, \delta, L)$ and $\varphi \in \mathbf{CTL}$

$\mathbf{SMC}(\varphi)$ computes $\llbracket \varphi \rrbracket = \{q \in \mathbf{Q} \mid q \models \varphi\}$

BDD-based Model Checking Algorithm

Given: $\mathbf{A} = (\mathbf{Q}, q_0, \delta, L)$ and $\varphi \in \mathbf{CTL}$

$\mathbf{SMC}(\varphi)$ computes $\llbracket \varphi \rrbracket = \{q \in \mathbf{Q} \mid q \models \varphi\}$

$\mathbf{SMC-BDD}(\varphi)$ computes $\mathbf{BDD}(\llbracket \varphi \rrbracket)$

BDD-based Model Checking Algorithm

Given: $A = (Q, q_0, \delta, L)$ and $\varphi \in CTL$

$SMC(\varphi)$ computes $\llbracket \varphi \rrbracket = \{q \in Q \mid q \models \varphi\}$

$SMC\text{-}BDD(\varphi)$ computes $BDD(\llbracket \varphi \rrbracket)$

$$A \models \varphi \iff q_0 \models \varphi \iff q_0 \in \llbracket \varphi \rrbracket$$

BDD-based Model Checking Algorithm

Given: $A = (Q, q_0, \delta, L)$ and $\varphi \in CTL$

$SMC(\varphi)$ computes $\llbracket \varphi \rrbracket = \{q \in Q \mid q \models \varphi\}$

$SMC\text{-}BDD(\varphi)$ computes $BDD(\llbracket \varphi \rrbracket)$

$$A \models \varphi \iff q_0 \models \varphi \iff q_0 \in \llbracket \varphi \rrbracket$$

$$q_0 \in \llbracket \varphi \rrbracket \iff BDD(f_{q_0}) \otimes BDD(\llbracket \varphi \rrbracket) \neq BDD(False)$$